HOCHSCHULE KONSTANZ TECHNIK, WIRTSCHAFT UND GESTALTUNG (HTWG)
Fakultät Informatik
Rechner- und Kommunikationsnetze
Prof. Dr. Dirk Staehle

Vorlesung Rechnernetze
AIN 5

Laborübung

Paketanalyse mit Wireshark – Teil 1
(Teilaufgaben 3 bis 5)

Prof. Dr. Dirk Staehle

Die Abgabe erfolgt durch Hochladen der bearbeiteten Word-Datei in Moodle.

Bearbeitung in Zweier-Teams

Team-Mitglied 1:

Team-Mitglied 2:

3 Ruhezustand und HTTP im Klartext

In diesem Abschnitt erstellen Sie zwei Mitschnitte: einen, in dem Sie selbst nichts tun, und einen mit HTTP-Verkehr im Klartext.

Hinweis: Für diesen Teil gelten die Hinweise zur Bearbeitung und zum Datenschutz aus der Einleitung von Teil 1.1.

3.1 Mitschnitt 1: Ruhezustand

Ihr Rechner sendet auch dann Pakete, wenn Sie selbst nichts tun. Schließen Sie alle Anwendungen und starten Sie dann den Mitschnitt auf der Schnittstelle, über die Ihr Rechner mit dem Netz verbunden ist. Schneiden Sie den Verkehr so lange mit, bis Sie einige (>10) Pakete sehen. Das kann etwas dauern oder sehr schnell gehen, je nachdem, was Ihr Rechner gerade im Hintergrund macht.

1. Notieren Sie bis zu fünf Protokolle (Spalte „Protocol“ in Wireshark), die Ihr Rechner nutzt, und recherchieren Sie, was diese Protokolle machen und welcher Protokollschicht sie zugeordnet werden. Mit welcher IP-Adresse haben diese Protokolle kommuniziert?

2. Welche IPv4- und ggf. IPv6-Adresse sowie welche MAC-Adresse hat Ihr Rechner laut Mitschnitt? Vergleichen Sie mit der Ausgabe von `ipconfig /all` (Windows) bzw. `ip addr` (Linux) oder `ifconfig` (macOS).

3. An welche Ziel-MAC-Adresse sendet Ihr Rechner Pakete? Vergleichen Sie die Ziel-MAC-Adresse für verschiedene Ziel-IP-Adressen. Welchem Netzknoten können Sie die Ziel-MAC-Adresse zuordnen?

DHCP
ICMP
HTTP
TCP
UDP/XML

DHCP(Application Layer)

DHCP ermöglicht es, angeschlossene Clients ohne manuelle Konfiguration der Netzschnittstelle in ein bestehendes Netz einzubinden. Nötige Informationen wie IP-Adresse, Netzmaske, Gateway, Name Server (DNS) und ggf. weitere Einstellungen werden automatisch vergeben, sofern das Betriebssystem des jeweiligen Clients dies unterstützt.

ICMP(Internet Layer)

ICMP ist Bestandteil von IPv4, wird aber wie ein eigenständiges Protokoll behandelt. Es wird von jedem Router und jedem Rechner erwartet, dass sie ICMP „verstehen“. Die meisten ICMP-Pakete enthalten Diagnose-Informationen: Sie werden vom Router zur Quelle zurückgeschickt, wenn der Router Pakete verwirft, etwa weil beispielsweise das Ziel nicht erreichbar ist oder die TTL abgelaufen ist. Es gelten folgende Grundsätze:

> Dieser Abschnitt ist nicht hinreichend mit Belegen (beispielsweise Einzelnachweisen) ausgestattet. Angaben ohne ausreichenden Beleg könnten demnächst entfernt werden. Bitte hilf Wikipedia, indem du die Angaben recherchierst und gute Belege einfügst.
> *Laut https://www.sei.cmu.edu/documents/505/1998_019_001_496180.pdf [605 KB] gibts per ICMP sehr wohl Antworten auf Broadcasts.*

- ICMP benutzt IP als Kommunikationsbasis, indem es sich selbst als Protokoll einer höheren Schicht interpretiert, d. h. ICMP-Nachrichten werden in IP-Paketen gekapselt.
- ICMP erkennt einige Fehlerzustände, macht aber IPv4 nicht zu einem zuverlässigen Protokoll.
- ICMP analysiert Fehler in jedem IP-Paket, mit Ausnahme solcher, die eine ICMP-Nachricht tragen.
- ICMP-Nachrichten werden nicht als Antwort auf Pakete an Zieladressen versendet, bei denen es sich um Multicast- oder Broadcast-Adressen handelt.
- ICMP-Nachrichten antworten nur einer eindeutigen Quell-IP-Adresse.

HTTP(Application Layer)

HTTP (**Hypertext Transfer Protocol**) is an application layer protocol in the Internet protocol suite for distributed, collaborative, hypermedia information systems.[1] HTTP is the foundation of data communication for the World Wide Web, where hypertext documents include hyperlinks to other resources that the user can easily access, for example by a mouse click or by tapping the screen in a web browser.

TCP(Transport Layer)

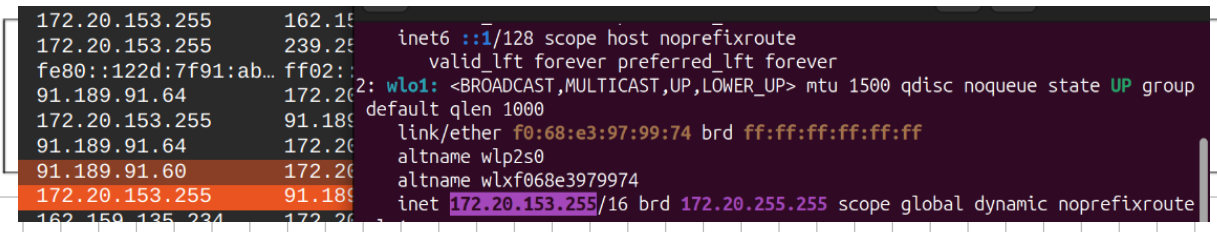
The **Transmission Control Protocol** (**TCP**) is one of the main protocols of the Internet protocol suite, providing reliable, ordered, and error-checked delivery of a stream of octets (bytes) between applications running on hosts communicating via an IP network. It originated in the initial network implementation in which it complemented the Internet Protocol (IP). Therefore, the entire suite is commonly referred to as TCP/IP.

UDP/XML(Transport Layer)

In computer networking, the **User Datagram Protocol** (**UDP**) is one of the core communication protocols of the Internet protocol suite used to send messages (transported as datagrams in packets) to other hosts on an Internet Protocol (IP) network. Within an IP network, UDP does not require prior communication to set up communication channels or data paths.

XML da eine XML Datei inkludiert ist ??

2. Welche IPv4- und ggf. IPv6-Adresse sowie welche MAC-Adresse hat Ihr Rechner laut Mitschnitt? Vergleichen Sie mit der Ausgabe von `ipconfig /all` (Windows) bzw. `ip addr` (Linux) oder `ifconfig` (macOS).



3. An welche Ziel-MAC-Adresse sendet Ihr Rechner Pakete? Vergleichen Sie die Ziel-MAC-Adresse für verschiedene Ziel-IP-Adressen. Welchem Netzknoten können Sie die Ziel-MAC-Adresse zuordnen?

5	0.438789273	Cisco_f4:9b:4d	AzureWaveTec_97:99…	ARP

Zugriff auf Ethernet des Laptops

14	1.004974201	AzureWaveTec_97:99…	LANCOM_48:0b:14	EAPOL

Zugriff auf Eduroam(vermutlich)

4	0.432011637	AzureWaveTec_97:99…	Broadcast	ARP
5	0.438789273	Cisco_f4:9b:4d	AzureWaveTec_97:99…	ARP

Anfrage und Antwort auf Eduroam

3.2 Mitschnitt 2: HTTP im Klartext

Starten Sie einen neuen Mitschnitt und erzeugen Sie HTTP-Verkehr nach einer der folgenden Varianten (gerne auch mehreren). Beenden und speichern Sie den Mitschnitt anschließend. Die meisten Webseiten werden heute nur noch verschlüsselt ausgeliefert, und Browser versuchen zunehmend, automatisch auf HTTPS umzustellen. Die Varianten sind so gewählt, dass Sie trotzdem unverschlüsselten Verkehr erhalten.

Variante A: Browser. Rufen Sie im Browser die folgende Seite auf. Geben Sie `http://` ausdrücklich mit ein.

```
http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html
```

Aktuelle Browser versuchen zunächst automatisch eine verschlüsselte Verbindung aufzubauen und warnen teilweise vor unverschlüsselten Seiten. So erzwingen Sie HTTP:

- **Chrome:** Seit Version 154 warnt Chrome vor dem Aufruf öffentlicher Seiten über HTTP. Klicken Sie in der Warnung auf „Weiter zur Website“ oder deaktivieren Sie unter `chrome://settings/security` die Option „Immer sichere Verbindungen verwenden“.
- **Edge:** Suchen Sie in den Einstellungen nach „HTTPS“ und deaktivieren Sie „Automatisches HTTPS“ bzw. den automatischen Wechsel zu sichereren Verbindungen.
- **Firefox:** Firefox versucht seit Version 136 zunächst HTTPS und fällt auf HTTP zurück, wenn der Server kein HTTPS anbietet. Falls Sie den „Nur-HTTPS-Modus“ aktiviert haben (Einstellungen → Datenschutz & Sicherheit), schalten Sie ihn für die Übung aus.
- **Safari:** Wenn unter Safari → Einstellungen → Sicherheit „Vor dem Verbinden mit einer Website über HTTP warnen“ aktiviert ist, bestätigen Sie die Warnung.

Variante B: curl. Rufen Sie dieselbe Seite im Terminal bzw. in der Eingabeaufforderung ab. curl stellt nie automatisch auf HTTPS um.

```
curl http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html
```

Variante C: Eigener Webserver auf Ihrem Rechner. Starten Sie in einem beliebigen Verzeichnis einen Webserver (unter Windows ggf. `python` oder `py` statt `python3`):

```
python3 -m http.server 8000
```

Rufen Sie im Browser auf demselben Rechner `http://127.0.0.1:8000` auf. Dieser Verkehr verlässt Ihren Rechner nicht. Sie müssen in Wireshark deshalb die Loopback-Schnittstelle mitschneiden statt der WLAN-Schnittstelle: unter Windows „Adapter for loopback traffic capture“, unter macOS „lo0“, unter Linux „lo“.

Kurz erklärt: Request, Response und Verbindung. Beim Abruf einer Webseite schickt Ihr Rechner (der *Client*) eine Anfrage an den Webserver, den **HTTP-Request** (z. B. `GET /index.html`). Der Server antwortet mit der **HTTP-Response**, die die Seite enthält.

Bevor Ihr Rechner den Request senden kann, muss er zwei Dinge erledigen: Er muss die IP-Adresse des Servers kennen; falls er sie noch nicht kennt, fragt er sie per **DNS** beim Nameserver ab. Außerdem baut er eine **TCP-Verbindung** zum Server auf. Eine Verbindung ist eine

Vereinbarung zwischen zwei Anwendungen, miteinander Daten auszutauschen: Zuerst stimmen sich beide Seiten mit drei kurzen Paketen ab (Verbindungsaufbau, in Wireshark als `SYN`, `SYN, ACK` und `ACK` angezeigt), dann werden die Daten übertragen, und am Ende wird die Verbindung mit `FIN`-Paketen wieder abgebaut. Wie das im Detail funktioniert, behandeln wir später in der Vorlesung.

4. Suchen Sie mit dem Filter `http` Ihren HTTP-Request (in der Spalte „Info" steht `GET …`) und merken Sie sich die Paketnummer. Entfernen Sie den Filter wieder und betrachten Sie die etwa zehn Pakete vor dem Request: Quell- und Ziel-IP-Adresse, Protokoll, Portnummern und Inhalt (Spalte „Info"). Welche dieser Pakete könnten mit Ihrem Request zusammenhängen, und was ist dort vermutlich passiert?
   Tipp: Achten Sie auf Pakete mit der IP-Adresse des Servers und auf DNS-Pakete mit dem Namen `gaia.cs.umass.edu`. Je nach Variante finden Sie nicht alles davon: Warum?

5. Öffnen Sie mit Rechtsklick auf den Request → Folgen → TCP-Stream die gesamte Verbindung. Wireshark zeigt den Inhalt der Verbindung in einem eigenen Fenster und in der Paketliste nur noch die Pakete dieser Verbindung. Finden Sie in der Paketliste den Verbindungsaufbau, den HTTP-Request, die HTTP-Response und den Verbindungsabbau (siehe Kasten oben). Aus wie vielen Paketen besteht die Verbindung?
   Hinweis: Browser halten Verbindungen oft noch eine Weile offen. Falls Sie keinen Verbindungsabbau finden, notieren Sie das einfach.

6. Bestimmen Sie aus den Zeitstempeln in der Spalte „Time", wie lange es vom Senden des HTTP-Requests bis zum Eintreffen der HTTP-Response gedauert hat. Wählen Sie anschließend die HTTP-Response aus und klappen Sie in den Paketdetails den HTTP-Teil auf. Welche Angabe finden Sie dort, die Ihre Rechnung bestätigt? *[Diskussion]* Einige Angaben stehen in eckigen Klammern. Was bedeutet das, und sind diese Angaben im Paket tatsächlich enthalten?

Vereinbarung zwischen zwei Anwendungen, miteinander Daten auszutauschen: Zuerst stimmen sich beide Seiten mit drei kurzen Paketen ab (Verbindungsaufbau, in Wireshark als `SYN`, `SYN, ACK` und `ACK` angezeigt), dann werden die Daten übertragen, und am Ende wird die Verbindung mit `FIN`-Paketen wieder abgebaut. Wie das im Detail funktioniert, behandeln wir später in der Vorlesung.

4. Suchen Sie mit dem Filter `http` Ihren HTTP-Request (in der Spalte „Info“ steht `GET …`) und merken Sie sich die Paketnummer. Entfernen Sie den Filter wieder und betrachten Sie die etwa zehn Pakete vor dem Request: Quell- und Ziel-IP-Adresse, Protokoll, Portnummern und Inhalt (Spalte „Info“). Welche dieser Pakete könnten mit Ihrem Request zusammenhängen, und was ist dort vermutlich passiert?
   Tipp: Achten Sie auf Pakete mit der IP-Adresse des Servers und auf DNS-Pakete mit dem Namen `gaia.cs.umass.edu`. Je nach Variante finden Sie nicht alles davon: Warum?

No.	Time	Delta	Source		Destination	Protocol	Lengtl	Info
1	0.000000000		162.159.134.234		172.20.155.130	TLSv1.2	159	Application Data
2	0.000078164	0.000078164	172.20.155.130		162.159.134.234	TCP	66	60012 → 443 [ACK] Seq=1 Ack=94 Win=1590 Len=0 TSval=3459934956 TSecr=3197928951
3	2.407084241	2.407006077	172.20.155.130	Request IP from DNS	141.37.11.2	DNS	88	Standard query 0x037c AAAA gaia.cs.umass.edu OPT
4	2.407271532	0.000187291	172.20.155.130		141.37.11.2	DNS	88	Standard query 0x100b A gaia.cs.umass.edu OPT
5	2.408235530	0.000963998	172.20.155.130	Start TCP Handshake	128.119.245.12	TCP	74	42954 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=4096954511 TSecr=0 WS=…
6	2.412714298	0.004478768	141.37.11.2	Receive Ip from DNS	172.20.155.130	DNS	104	Standard query response 0x100b A gaia.cs.umass.edu A 128.119.245.12 OPT
7	2.412714437	0.000000139	141.37.11.2		172.20.155.130	DNS	141	Standard query response 0x037c AAAA gaia.cs.umass.edu SOA unix1.cs.umass.edu OPT
8	2.662504343	0.249789906	128.119.245.12	Server Responds with ACK	172.20.155.130	TCP	74	80 → 42954 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1380 SACK_PERM TSval=4277990627 …
9	2.662571765	0.000067422	172.20.155.130	We responds with ACK	128.119.245.12	TCP	66	42954 → 80 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=4096954765 TSecr=4277990627
10	2.662736141	0.000164376	172.20.155.130	Request HTML from Server	128.119.245.12	HTTP	188	GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1
11	2.866696385	0.203960244	128.119.245.12		172.20.155.130	TCP	66	80 → 42954 [ACK] Seq=1 Ack=123 Win=65152 Len=0 TSval=4277990881 TSecr=4096954765
12	2.866696733	0.000000348	128.119.245.12	Server Responds with HTML	172.20.155.130	HTTP	451	HTTP/1.1 200 OK (text/html)
13	2.866778610	0.000081877	172.20.155.130		128.119.245.12	TCP	66	42954 → 80 [ACK] Seq=123 Ack=386 Win=64512 Len=0 TSval=4096954969 TSecr=4277990882
14	2.867115068	0.000336458	172.20.155.130	TCP finish ACK	128.119.245.12	TCP	66	42954 → 80 [FIN, ACK] Seq=123 Ack=386 Win=64512 Len=0 TSval=4096954970 TSecr=4277990882
15	3.071162341	0.204047273	128.119.245.12	TCP finish ACK	172.20.155.130	TCP	66	80 → 42954 [FIN, ACK] Seq=386 Ack=124 Win=65152 Len=0 TSval=4277991085 TSecr=4096954970
16	3.071228937	0.000066596	172.20.155.130	Finish TCP Handshake	128.119.245.12	TCP	66	42954 → 80 [ACK] Seq=124 Ack=387 Win=64512 Len=0 TSval=4096955174 TSecr=4277991085
17	3.117522904	0.046293967	162.159.134.234	Exchange application Data	172.20.155.130	TLSv1.2	144	Application Data
18	3.117586711	0.000063807	172.20.155.130		162.159.134.234	TCP	66	60012 → 443 [ACK] Seq=1 Ack=172 Win=1590 Len=0 TSval=3459938073 TSecr=3197932108
19	4.064273077	0.946686366	2001:7c0:5f0:f128:a64e:9f9:f50:66c9		2a00:1450:4001:c15::5f	UDP	91	38543 → 443 Len=29
20	4.104332200	0.040059123	2a00:1450:4001:c15::5f		2001:7c0:5f0:f128:a64e:9f9:f50:66c9	UDP	92	443 → 38543 Len=30
21	5.016829312	0.912497112	162.159.134.234	Exchang application data	172.20.155.130	TLSv1.2	133	Application Data
22	5.016884459	0.000055147	172.20.155.130		162.159.134.234	TCP	66	60012 → 443 [ACK] Seq=1 Ack=239 Win=1590 Len=0 TSval=3459939972 TSecr=3197933995

5. Öffnen Sie mit Rechtsklick auf den Request → Folgen → TCP-Stream die gesamte Verbindung. Wireshark zeigt den Inhalt der Verbindung in einem eigenen Fenster und in der Paketliste nur noch die Pakete dieser Verbindung. Finden Sie in der Paketliste den Verbindungsaufbau, den HTTP-Request, die HTTP-Response und den Verbindungsabbau (siehe Kasten oben). Aus wie vielen Paketen besteht die Verbindung?
   Hinweis: Browser halten Verbindungen oft noch eine Weile offen. Falls Sie keinen Verbindungsabbau finden, notieren Sie das einfach.

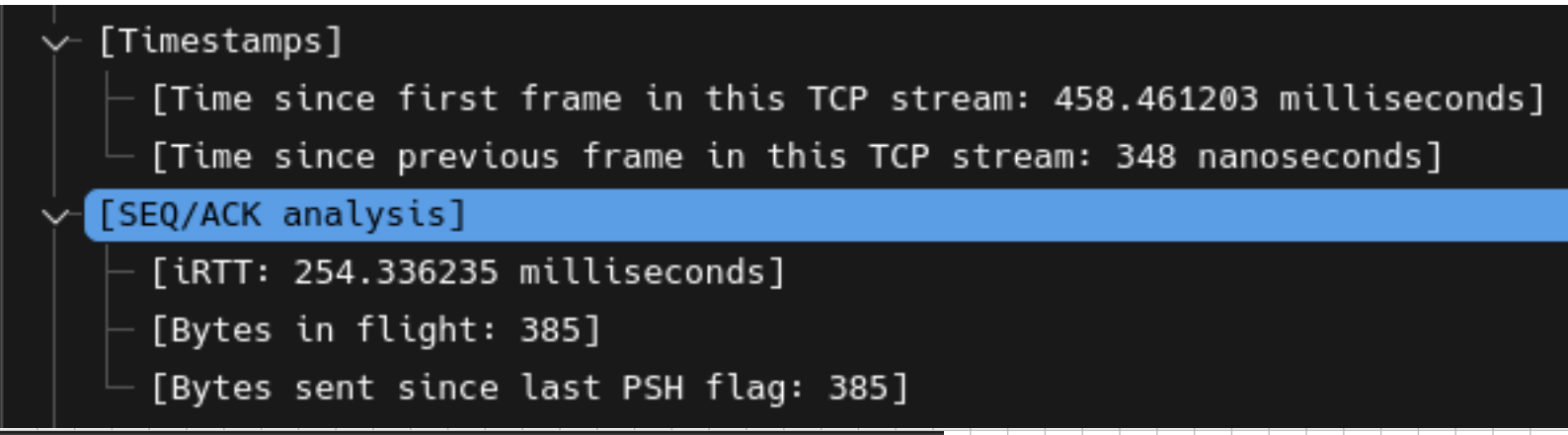
```
GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1
Host: gaia.cs.umass.edu
User-Agent: curl/8.23.0
Accept: */*

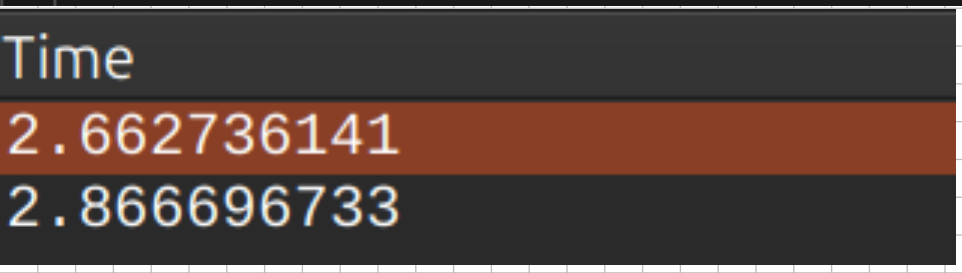

HTTP/1.1 200 OK
Date: Wed, 21 Oct 2026 14:41:35 GMT
Server: Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.5 mod_fcgid/2.3.9 mod_perl/2.0.
12 Perl/v5.32.1
Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT
ETag: "51-64231b6715777"
Accept-Ranges: bytes
Content-Length: 81
Content-Type: text/html; charset=UTF-8

<html>
Congratulations!  You've downloaded the first Wireshark lab file!
</html>
```

HTTP Request mit GET Aufbau mit ‹html› abbau mit ‹/html›
TCP Kontakt besteht aus 2 Paketen

6. Bestimmen Sie aus den Zeitstempeln in der Spalte „Time“, wie lange es vom Senden des HTTP-Requests bis zum Eintreffen der HTTP-Response gedauert hat. Wählen Sie anschließend die HTTP-Response aus und klappen Sie in den Paketdetails den HTTP-Teil auf. Welche Angabe finden Sie dort, die Ihre Rechnung bestätigt? *[Diskussion]* Einige Angaben stehen in eckigen Klammern. Was bedeutet das, und sind diese Angaben im Paket tatsächlich enthalten?





Dauer antwort = 0.203960592 millisekunden

7. Vergleichen Sie Ihren HTTP-Request mit dem Paket aus dem Hex-Dump in Abschnitt 2 in drei Punkten: Ziel-Port, Zeile `Host:` und Zeile `User-Agent:` im HTTP-Teil. Was verrät der User-Agent jeweils über das Programm (und ggf. das Betriebssystem), das den Request geschickt hat?

```
Destination Port: 80
0000  00 a6 ca f4 9b 4d 4c 1d  96 e6 1b 62 08 00 45 00   ·····ML· ···b··E·
0010  00 ae 8f 06 40 00 40 06  ee 28 ac 14 9b 82 80 77   ····@·@· ·(·····w
0020  f5 0c a7 ca 00 50 b7 e6  51 40 f8 a3 39 7a 80 18   ·····P·· Q@··9z··
0030  00 3f bd bb 00 00 01 01  08 0a f4 32 91 8d fe fc   ·?······ ···2····
0040  f4 e3 47 45 54 20 2f 77  69 72 65 73 68 61 72 6b   ··GET /w ireshark
0050  2d 6c 61 62 73 2f 49 4e  54 52 4f 2d 77 69 72 65   -labs/IN TRO-wire
0060  73 68 61 72 6b 2d 66 69  6c 65 31 2e 68 74 6d 6c   shark-fi le1.html
0070  20 48 54 54 50 2f 31 2e  31 0d 0a 48 6f 73 74 3a    HTTP/1. 1··Host:
0080  20 67 61 69 61 2e 63 73  2e 75 6d 61 73 73 2e 65    gaia.cs .umass.e
0090  64 75 0d 0a 55 73 65 72  2d 41 67 65 6e 74 3a 20   du··User -Agent:
00a0  63 75 72 6c 2f 38 2e 32  33 2e 30 0d 0a 41 63 63   curl/8.2 3.0··Acc
00b0  65 70 74 3a 20 2a 2f 2a  0d 0a 0d 0a               ept: */* ····
```

Beide haben gleichen Destination Port: 80

```
0000  38 22 d6 67 19 00 00 21  cc 63 82 2c 08 00 45 00   8".g...!.c.,..E.
0010  02 9c 02 ed 40 00 80 06  40 66 8d 25 1d 5d 5b c6   ....@...@f.%.][.
0020  ae c0 e2 26 00 50 4f 4c  29 24 72 ce 3c d4 50 18   ...&.POL)$r.<.P.
0030  40 b0 62 e7 00 00 47 45  54 20 2f 77 69 6b 69 2f   @.b...GET /wiki/
0040  53 69 6d 70 6c 65 5f 53  65 72 76 69 63 65 5f 44   Simple_S ervice_D
0050  69 73 63 6f 76 65 72 79  5f 50 72 6f 74 6f 63 6f   iscovery _Protoco
0060  6c 20 48 54 54 50 2f 31  2e 31 0d 0a 48 6f 73 74   l HTTP/1 .1..Host
0070  3a 20 64 65 2e 77 69 6b  69 70 65 64 69 61 2e 6f   : de.wik ipedia.o
0080  72 67 0d 0a 55 73 65 72  2d 41 67 65 6e 74 3a 20   rg..User -Agent:
0090  4d 6f 7a 69 6c 6c 61 2f  35 2e 30 20 28 57 69 6e   Mozilla/ 5.0 (Win
00a0  64 6f 77 73 20 4e 54 20  36 2e 31 3b 20 57 4f 57   dows NT  6.1; WOW
00b0  36 34 3b 20 72 76 3a 33  32 2e 30 29 20 47 65 63   64; rv:3 2.0) Gec
```

```
User-Agent: curl/8.23.0\r\n
0000  00 a6 ca f4 9b 4d 4c 1d  96 e6 1b 62 08 00 45 00   ·····ML· ···b··E·
0010  00 ae 8f 06 40 00 40 06  ee 28 ac 14 9b 82 80 77   ····@·@· ·(·····w
0020  f5 0c a7 ca 00 50 b7 e6  51 40 f8 a3 39 7a 80 18   ·····P·· Q@··9z··
0030  00 3f bd bb 00 00 01 01  08 0a f4 32 91 8d fe fc   ·?······ ···2····
0040  f4 e3 47 45 54 20 2f 77  69 72 65 73 68 61 72 6b   ··GET /w ireshark
0050  2d 6c 61 62 73 2f 49 4e  54 52 4f 2d 77 69 72 65   -labs/IN TRO-wire
0060  73 68 61 72 6b 2d 66 69  6c 65 31 2e 68 74 6d 6c   shark-fi le1.html
0070  20 48 54 54 50 2f 31 2e  31 0d 0a 48 6f 73 74 3a    HTTP/1. 1··Host:
0080  20 67 61 69 61 2e 63 73  2e 75 6d 61 73 73 2e 65    gaia.cs .umass.e
0090  64 75 0d 0a 55 73 65 72  2d 41 67 65 6e 74 3a 20   du··User -Agent:
00a0  63 75 72 6c 2f 38 2e 32  33 2e 30 0d 0a 41 63 63   curl/8.2 3.0··Acc
00b0  65 70 74 3a 20 2a 2f 2a  0d 0a 0d 0a               ept: */* ····
```

User Agen: curl

User agent: Mozilla Firefox

```
0000  38 22 d6 67 19 00 00 21  cc 63 82 2c 08 00 45 00   8".g...!.c.,..E.
0010  02 9c 02 ed 40 00 80 06  40 66 8d 25 1d 5d 5b c6   ....@...@f.%.][.
0020  ae c0 e2 26 00 50 4f 4c  29 24 72 ce 3c d4 50 18   ...&.POL)$r.<.P.
0030  40 b0 62 e7 00 00 47 45  54 20 2f 77 69 6b 69 2f   @.b...GET /wiki/
0040  53 69 6d 70 6c 65 5f 53  65 72 76 69 63 65 5f 44   Simple_S ervice_D
0050  69 73 63 6f 76 65 72 79  5f 50 72 6f 74 6f 63 6f   iscovery _Protoco
0060  6c 20 48 54 54 50 2f 31  2e 31 0d 0a 48 6f 73 74   l HTTP/1 .1..Host
0070  3a 20 64 65 2e 77 69 6b  69 70 65 64 69 61 2e 6f   : de.wik ipedia.o
0080  72 67 0d 0a 55 73 65 72  2d 41 67 65 6e 74 3a 20   rg..User -Agent:
0090  4d 6f 7a 69 6c 6c 61 2f  35 2e 30 20 28 57 69 6e   Mozilla/ 5.0 (Win
00a0  64 6f 77 73 20 4e 54 20  36 2e 31 3b 20 57 4f 57   dows NT  6.1; WOW
00b0  36 34 3b 20 72 76 3a 33  32 2e 30 29 20 47 65 63   64; rv:3 2.0) Gec
```

```
    Host: gaia.cs.umass.edu\r\n
    User-Agent: curl/8.23.0\r\n
    Accept: */*\r\n
    \r\n
0000  00 a6 ca f4 9b 4d 4c 1d  96 e6 1b 62 08 00 45 00   .....ML. ...b..E.
0010  00 ae 8f 06 40 00 40 06  ee 28 ac 14 9b 82 80 77   ....@.@. .(.....w
0020  f5 0c a7 ca 00 50 b7 e6  51 40 f8 a3 39 7a 80 18   .....P.. Q@..9z..
0030  00 3f bd bb 00 00 01 01  08 0a f4 32 91 8d fe fc   .?...... ...2....
0040  f4 e3 47 45 54 20 2f 77  69 72 65 73 68 61 72 6b   ..GET /w ireshark
0050  2d 6c 61 62 73 2f 49 4e  54 52 4f 2d 77 69 72 65   -labs/IN TRO-wire
0060  73 68 61 72 6b 2d 66 69  6c 65 31 2e 68 74 6d 6c   shark-fi le1.html
0070  20 48 54 54 50 2f 31 2e  31 0d 0a 48 6f 73 74 3a    HTTP/1. 1..Host:
0080  20 67 61 69 61 2e 63 73  2e 75 6d 61 73 73 2e 65    gaia.cs .umass.e
0090  64 75 0d 0a 55 73 65 72  2d 41 67 65 6e 74 3a 20   du..User -Agent:
00a0  63 75 72 6c 2f 38 2e 32  33 2e 30 0d 0a 41 63 63   curl/8.2 3.0..Acc
00b0  65 70 74 3a 20 2a 2f 2a  0d 0a 0d 0a               ept: */* ....
```

Host: Gaia

Host wikipedia

```
0000  38 22 d6 67 19 00 00 21  cc 63 82 2c 08 00 45 00   8".g...!.c.,..E.
0010  02 9c 02 ed 40 00 80 06  40 66 8d 25 1d 5d 5b c6   ....@...@f.%.][.
0020  ae c0 e2 26 00 50 4f 4c  29 24 72 ce 3c d4 50 18   ...&.POL)$r.<.P.
0030  40 b0 62 e7 00 00 47 45  54 20 2f 77 69 6b 69 2f   @.b...GE T /wiki/
0040  53 69 6d 70 6c 65 5f 53  65 72 76 69 63 65 5f 44   Simple_S ervice_D
0050  69 73 63 6f 76 65 72 79  5f 50 72 6f 74 6f 63 6f   iscovery _Protoco
0060  6c 20 48 54 54 50 2f 31  2e 31 0d 0a 48 6f 73 74   l HTTP/1 .1..Host
0070  3a 20 64 65 2e 77 69 6b  69 70 65 64 69 61 2e 6f   : de.wik ipedia.o
0080  72 67 0d 0a 55 73 65 72  2d 41 67 65 6e 74 3a 20   rg..User -Agent:
0090  4d 6f 7a 69 6c 6c 61 2f  35 2e 30 20 28 57 69 6e   Mozilla/ 5.0 (Win
00a0  64 6f 77 73 20 4e 54 20  36 2e 31 3b 20 57 4f 57   dows NT  6.1; WOW
00b0  36 34 3b 20 72 76 3a 33  32 2e 30 29 20 47 65 63   64; rv:3 2.0) Gec
```

7. Vergleichen Sie Ihren HTTP-Request mit dem Paket aus dem Hex-Dump in Abschnitt 2 in drei Punkten: Ziel-Port, Zeile `Host:` und Zeile `User-Agent:` im HTTP-Teil. Was verrät der User-Agent jeweils über das Programm (und ggf. das Betriebssystem), das den Request geschickt hat?

4 Filter

Display-Filter haben Sie in den bisherigen Aufgaben bereits benutzt (z. B. `icmp`, `dns` und `http`). In diesem Abschnitt vertiefen Sie den Umgang damit.

Wireshark kennt zwei Arten von Filtern. Capture-Filter legen fest, welche Pakete überhaupt aufgezeichnet werden. Display-Filter legen fest, welche der aufgezeichneten Pakete angezeigt werden. Beide haben eine unterschiedliche Syntax.

Display-Filter filtern nach Protokollen und nach dem Inhalt von Header-Feldern. Der Filter `ip.addr==192.140.168.15` zeigt alle Pakete mit dieser Quell- oder Zieladresse, `udp.port==1500` alle Pakete mit UDP-Port 1500 auf Sender- oder Empfängerseite. Bedingungen lassen sich mit `&&`, `||` und `!` verknüpfen, Werte mit `==`, `!=`, `<`, `>` oder `contains` vergleichen. Hilfreich sind die Autovervollständigung im Filterfeld (z. B. nach Eingabe von `tcp.`) und die Übersicht unter https://www.wireshark.org/docs/dfref/.

Schneiden Sie 30 bis 60 Sekunden lang mit, während Sie einige verschiedene Webseiten aufrufen (z. B. eine Suchmaschine, ein Video-Portal, eine Nachrichtenseite).

1. Formulieren Sie für die folgenden Aufgaben jeweils einen Display-Filter und notieren Sie, wie viele Pakete er anzeigt. Tipp: Klicken Sie in den Paketdetails auf das gesuchte Feld. Den Feldnamen sehen Sie in der Statusleiste, und mit Rechtsklick → Als Filter anwenden übernehmen Sie ihn in das Filterfeld.

Aufgabe	**Filter**	**Anzahl Pakete**
Alle Pakete, die an Ihren Rechner gerichtet sind (IPv4 und IPv6)	(ip || ipv6) && ip.addr==10.48.182.223	9791
Alle DNS-Anfragen, aber keine DNS-Antworten (Tipp: Feld `dns.flags.response`)	dns && dns.flags.response	122
Alle DNS-Anfragen nach einem Namen, der den Namen einer von Ihnen besuchten Seite enthält (Tipp: Operator `contains`)	dns.qry.name contains "wikipedia"	6

Aufgabe	Filter	Anzahl Pakete
Alle Pakete, die größer als 1000 Byte sind und an Ihren Rechner gehen	(ip || ipv6) && ip.addr==10.48.182.223 && dns.qry.len > 1000	0

2. Öffnen Sie Ihren Mitschnitt mit HTTP-Verkehr aus Abschnitt 3. Vergleichen Sie die Ergebnisse der Filter `http` und `tcp.port==80`. Welche Pakete zeigt nur der zweite Filter an, und warum? *[Diskussion]* Was beobachten Sie, wenn Sie in Abschnitt 3 Variante C (Port 8000) genutzt haben?

5 Verschlüsselter HTTP-Verkehr (HTTPS)

HTTPS-Verkehr läuft entweder über TCP (HTTP/1.1 und HTTP/2, verschlüsselt mit TLS) oder über UDP (HTTP/3 mit QUIC), beides standardmäßig auf Port 443. Starten Sie einen neuen Mitschnitt und rufen Sie im Browser nacheinander `https://www.htwg-konstanz.de` und `https://www.google.com` auf; laden Sie jede Seite zweimal.

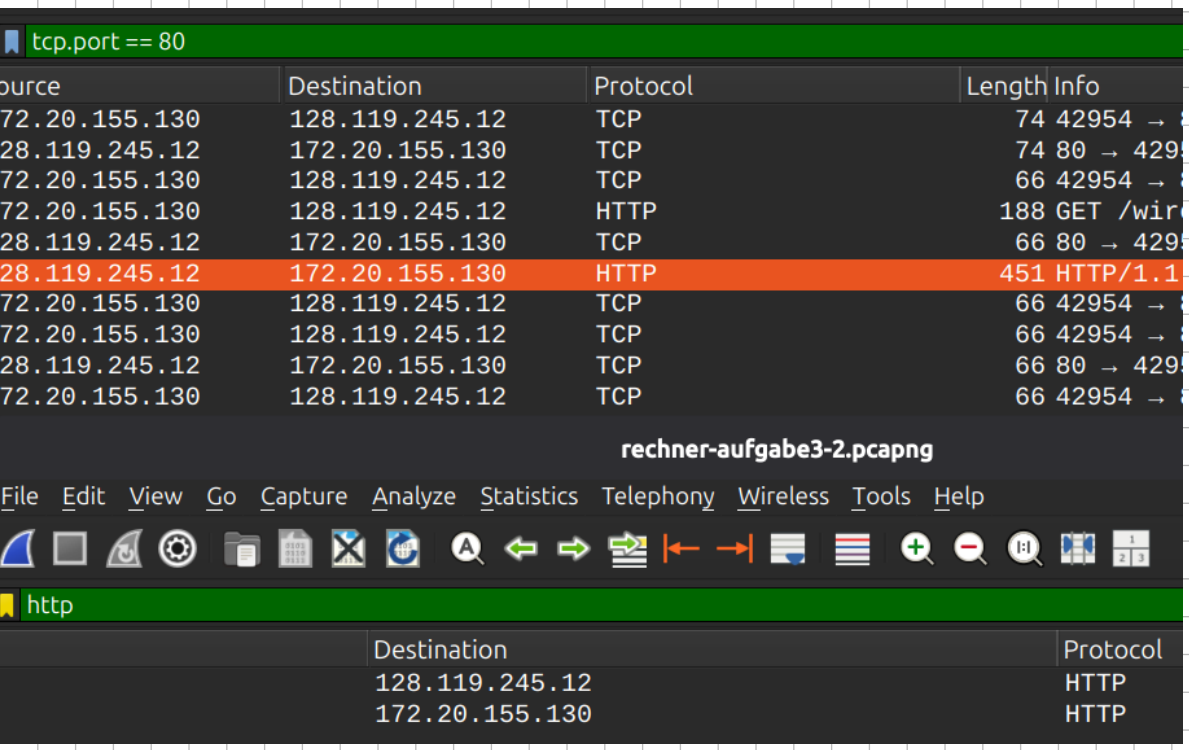
5.1 Analyse von verschlüsseltem Verkehr

1. Wie viele Pakete zeigen die Filter `tcp.port==443`, `udp.port==443`, `tls` und `quic` jeweils an? Welche der beiden Seiten nutzt QUIC? Welcher Filter erfasst den gesamten HTTPS-Verkehr?

2. *[Diskussion]* Es gibt einen Filter `http`, aber keinen Filter `https`. Warum?

3. Suchen Sie mit dem Filter `tls.handshake.type==1` die Client-Hello-Pakete, also jeweils das erste Paket des TLS-Handshakes. Wählen Sie das Client Hello an www.htwg-konstanz.de aus. Den Servernamen finden Sie in den Paketdetails unter Transport Layer Security → Handshake Protocol → Extension: server_name (SNI). Notieren Sie Ziel-IP-Adresse, Ziel-Port und Servernamen.

2. Öffnen Sie Ihren Mitschnitt mit HTTP-Verkehr aus Abschnitt 3. Vergleichen Sie die Ergebnisse der Filter `http` und `tcp.port==80`. Welche Pakete zeigt nur der zweite Filter an, und warum? *[Diskussion]* Was beobachten Sie, wenn Sie in Abschnitt 3 Variante C (Port 8000) genutzt haben?



Query nach tcp.port == 80 gibt alles was an diesem port ankommt aus was auch TCP beinhaltet
HTTP gibt strikt die HTTP Protokolle aus

5.1 Analyse von verschlüsseltem Verkehr

1. Wie viele Pakete zeigen die Filter `tcp.port==443`, `udp.port==443`, `tls` und `quic` jeweils an? Welche der beiden Seiten nutzt QUIC? Welcher Filter erfasst den gesamten HTTPS-Verkehr?

tcp.port == 443 ==›298 pakete
udp.port == 443 ==› 1420
quic ==›1420
tls ==›124 ==› 124

Seite mit quic:
Google nutzt quick

Filter der alles erfasst : udp v tcp

4. Füllen Sie die folgende Tabelle für einen Beobachter im Netz aus:

Information	**sichtbar (ja / nein / teilweise)**	**woran erkennbar?**
IP-Adresse des Servers	ja	src/dest
Ports und Transportprotokoll	ja	ports/protocoll
Domainname (z. B. www.beispiel.de)	teilweise	dns ja z.b. quic nicht
Vollständige URL inkl. Pfad	nein	
HTTP-Header (z. B. User-Agent, Cookies)	ja	in paketinfos
Inhalt der Seite	teilweise	teile sind lesbar
Übertragene Datenmenge	ja	UDP payload
Zeitpunkte und Dauer des Abrufs	ja	time

5.2 Entschlüsselung des eigenen HTTPS-Verkehrs

Firefox und Chrome-basierte Browser können die TLS-Sitzungsschlüssel in eine Datei schreiben, wenn die Umgebungsvariable `SSLKEYLOGFILE` gesetzt ist. Wireshark kann damit den Verkehr entschlüsseln. Safari unterstützt das nicht; nutzen Sie für diesen Abschnitt Firefox, Chrome oder Edge.

ACHTUNG: Sobald die Entschlüsselung eingerichtet ist, zeigt Wireshark alles im Klartext an, was Ihr Browser sendet und empfängt, also auch Passwörter, Cookies und Login-Daten, und zwar auch von Tabs, die nebenbei geöffnet sind. Melden Sie sich während der Übung nirgends an und nutzen Sie ein privates Fenster. Mit der Datei `keys.log` und einem Mitschnitt kann jeder Ihren Verkehr entschlüsseln. Geben Sie beides nicht weiter und laden Sie es nicht hoch. **Entfernen Sie nach der Übung die Umgebungsvariable und löschen Sie die Datei.**

- **Windows:** Suchen Sie im Startmenü nach „Umgebungsvariablen für dieses Konto bearbeiten“ und legen Sie unter Benutzervariablen eine neue Variable `SSLKEYLOGFILE` mit einem Pfad wie `C:\Users\<name>\keys.log` an.
- **macOS/Linux:** Setzen Sie im Terminal `export SSLKEYLOGFILE=~/keys.log` und starten Sie den Browser aus demselben Terminal, z. B. mit `firefox &` (Linux) oder `/Applications/Firefox.app/Contents/MacOS/firefox &` (macOS).
- **Browser neu starten:** Beenden Sie den Browser vollständig und starten Sie ihn neu. Chrome und Edge laufen unter Windows oft im Hintergrund weiter; beenden Sie sie dann im Task-Manager. Rufen Sie eine Seite auf und prüfen Sie, dass die Datei `keys.log` entstanden ist und wächst.
- **Wireshark:** Tragen Sie unter Bearbeiten → Einstellungen → Protocols → TLS im Feld „(Pre)-Master-Secret log filename“ dieselbe Datei ein (unter macOS: Wireshark → Einstellungen).

5. Starten Sie einen neuen Mitschnitt und rufen Sie die beiden Seiten im privaten Fenster erneut auf. Welche Protokolle zeigt Wireshark jetzt oberhalb von TLS bzw. QUIC an? Finden Sie einen Request, der den vollständigen Pfad einer abgerufenen Ressource enthält? Fügen Sie einen Screenshot ein.

zeigt über tls und quic immer wieder udp an

6. *[Diskussion]* Warum funktioniert die Entschlüsselung nur auf Ihrem eigenen Rechner und nicht für einen Beobachter im Netz? Was würde es bedeuten, wenn jemand Zugriff auf Ihre Datei keys.log hätte? Löschen Sie die Umgebungsvariable und die Datei nach der Übung wieder.

mit der datei können gesendete informationen verschlüsselt werden, ein beobachter im netz ohne die datei kann diese nicht entschlüsseln, außer er hätte die Datei zu entschlüsselung